Mini-Assignment #4: Introduction to Prediction

S022 S2026

Introduction

This mini-assignment is a **prediction competition**. You are given three datasets (A, B, and C), each containing a predictor variable X and an outcome variable Y. For each dataset, you are also given a **test set** that contains only X — your job is to predict the missing Y values as accurately as possible.

There is no single right answer here. The goal is to think carefully about the relationship between X and Y in each dataset, choose a sensible modeling approach, and produce predictions that hold up on data you have not seen.

The Data

Each dataset is a CSV file in the `data/` folder of your project.

File	Contents
`datasetA.csv`	Training data for Dataset A (X and Y)
`datasetB.csv`	Training data for Dataset B (X and Y)
`datasetC.csv`	Training data for Dataset C (X and Y)
`testA.csv`	Test data for Dataset A (X only — predict Y)
`testB.csv`	Test data for Dataset B (X only — predict Y)
`testC.csv`	Test data for Dataset C (X only — predict Y)

Start by plotting each training dataset. Look at the relationship between X and Y before you decide on a model — the right approach will differ across the three datasets.

What You Need to Do

For **each of the three datasets**, you must do two things:

1. Generate predictions for the test set. Fit a model to the training data and use it to predict Y for every row in the corresponding test file. Use the `format_answer()` function in the skeleton script to print out your predictions, then paste them into the Google Form at the link below.

2. Estimate your predictive accuracy (RMSE). You do not have the true Y values for the test set, so you cannot directly check how well you did. Instead, use a **train/validation split** on the training data to get an estimate of your out-of-sample RMSE *before* submitting. Hold out roughly half your training data as a validation set, fit your model on the remaining half, predict on the validation set, and compute RMSE:

$$\widehat{\text{RMSE}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{Y}_i - Y_i)^2}$$

Report this estimated RMSE for each dataset in your submission. You can try the split more than once (with different random seeds) to get a sense of how stable your estimate is.

The skeleton script (`data_challenge_skeleton.R`) shows you how to do both of these steps for Dataset A. Adapt it for Datasets B and C.

Submitting Your Predictions

Submit your predictions using the Google Form here:

https://forms.gle/FcdkEx4Pyd6C6m9P9

Enter in a **secret team name** (you can work with your problem set group or individually, as you like) at the top of the form.

Then, for each dataset you will paste:

- Your **20 predicted Y values** (comma-separated, sorted by X), generated by `format_answer()`.
- Your **estimated RMSE** from the train/validation split.

Once submitted, submit your code to Canvas, putting a comment at the top of your code with your **secret team name**, so we know who is who! The secret team name means when we look at the answers in class, everyone will not be publicly known.

Grading

Each dataset is worth equal weight. Within each dataset, points are awarded as follows:

Component	Points	Criterion
Reasonable predictions	80%	Your predictions are sensible given the data (i.e., you engaged with the problem and produced predictions that are in the right ballpark).
Accurate RMSE estimate	10%	Your estimated RMSE is within 20% of the true RMSE we compute from the answer key.
Competitive predictions	10%	Your prediction error (RMSE on the test set) falls in the **top 20% of the class**.

A few notes on the grading:

- The 80% threshold is generous — if you plot the data, fit a reasonable model, and produce predictions that are not wildly off, you will get it.
- The RMSE estimate bonus rewards careful use of the train/validation split. Think about whether your held-out RMSE is likely to be an over- or under-estimate of the true test RMSE, and whether your split size might be affecting it.
- The top-20% bonus rewards finding the *right* model for each dataset. The three datasets were designed with different underlying structures, so the best approach will not be the same for all three.

Hints and Suggestions

- **Simple can be good.** For some datasets, a very simple model will actually beat a complex one. Overfitting is a real risk, especially in small samples.
- **Consider nonlinear models.** `loess()` is a flexible smoother that can capture nonlinear patterns. The `span` argument controls smoothness — smaller values follow the data more closely.
- **The validation split is noisy.** With small datasets, your estimated RMSE can vary quite a bit from split to split. You may want to try several random splits and average the results.
- **Do not over-engineer.** Fancy models are not always better. Ask yourself: does this model actually fit the data better, or does it just fit the training data better?

Getting Started

Open `data_challenge_skeleton.R` in your project. It already loads the data, shows you how to fit a simple model for Dataset A, format predictions, and run a train/validation split. Work through it and adapt the code for Datasets B and C.

Good luck!